

raylib4Dreamcast and GLdc Performance Investigation

Executive summary

The highest-confidence conclusion is that the biggest renderer-wide win on Dreamcast is to increase the share of work that reaches GLdc through array-draw submission paths that can hit its fast path, while reducing the amount of raylib helper traffic that goes through OpenGL 1.1-style immediate calls and state-heavy tiny draws. The archived `raylib4Dreamcast` repository is mostly build/distribution scaffolding and now points to the newer console-port work, so the meaningful performance work is concentrated in raylib/rigl and GLdc/libGL rather than in the wrapper repo itself. ¹

Two facts drive most of the recommendation set. First, rigl's documented batching model is primarily for backends other than OpenGL 1.1; Dreamcast ports are using the OpenGL 1.1 backend, so you should not assume the same internal batching behavior that raylib gets on GL 3.3 or ES2. Second, GLdc's `glDrawArrays` / `glDrawElements` path has explicit fast-path code for quads, triangles, and indexed draws, but even that fast path still performs CPU-side transform and attribute copying before submission. That means the real target is not "more draw calls with nicer APIs," but "fewer state changes and fewer CPU-side vertex conversions per visible primitive." ²

Call flow

A condensed call-flow for the hot rendering path is:

```
game code
-> raylib helpers / Draw* APIs
-> rigl
  -> either OpenGL 1.1 style immediate calls or array draws
-> GLdc/libGL
  -> submitVertices()
  -> header generation if GPU state changed
  -> CPU transform / attribute gather / effects
  -> append to poly-list buffers
-> KOS / PVR
  -> list-sorted scene submission to TA / PVR
```

That map is supported directly by the current code organization: raylib exposes vertex-level `rlBegin` / `rlEnd` style calls alongside array-draw entry points such as `rlDrawVertexArrayElements`; GLdc's `glDrawArrays` and `glDrawElements` both feed into `submitVertices()`; `submitVertices()` allocates/extends submission buffers, may emit a new poly header when state is dirty, loads the transform

state, generates vertices, and applies effects; and KOS documents that the PVR wants primitive/list types grouped together, with direct rendering being fastest when your submission is already sorted by list type.

3

One practical implication is that there are really two separate optimization problems. The first is raylib-side API behavior: how often helpers change textures, matrices, and draw modes, and whether they reach GLdc as immediate-mode work or as client-array draws. The second is GLdc-side submission cost: how many times it must rebuild headers, transform vertices, copy attributes, and append to list buffers before KOS finally hands grouped data to the PVR. 4

Confirmed and likely bottlenecks

The most clearly confirmed bottleneck is **state churn causing fresh poly-header work and extra submission bookkeeping inside GLdc**. In `submitVertices()`, GLdc checks whether a header is required based on either an empty vector or dirty GPU state, grows its aligned vectors, optionally calls `apply_poly_header()`, marks the state clean, loads the current transform state, then generates vertices and applies T&L effects. `apply_poly_header()` itself rebuilds PVR-facing state from current GL state, including culling, depth test/write, shading mode, scissor, fog, and alpha/blend settings. That means frequent texture/blend/depth/material changes are not just “a bind”; they can force new header emission and more fragmented PVR lists. 5

The next confirmed bottleneck is **CPU work inside GLdc even on “fast” array paths**. GLdc’s `generate()` chooses a fast path when `ATTRIB_LIST.fast_path` is set, and it has specialized cases for array quads, array triangles, generic arrays, and indexed draws. But the indexed fast path still loops index-by-index, calls `TransformVertex(...)`, and copies UV/color/secondary UV/normal data into submission structures. In other words, fast path here means “less generic processing,” not zero-copy submission. Indexed rendering is therefore still paying a CPU gather/transform cost for every referenced vertex. 6

A major likely bottleneck is **raylib helper traffic that lands on the OpenGL 1.1 immediate-style path instead of the GLdc array path**. rgl exposes immediate-style vertex calls (`rlBegin`, `rlEnd`, `rlVertex*`, `rlTexCoord*`, `rlColor*`) as first-class API, and rgl’s own notes explain that the internal default batch is the abstraction used to mimic immediate mode on non-OpenGL-1.1 backends. On Dreamcast, which is using the OpenGL 1.1 backend for these console ports, that strongly suggests many helper functions will not enjoy the same batching behavior as desktop GL 3.3/ES2 builds. The current raylib helper layer also contains many procedural 3D helpers that repeatedly do `rlPushMatrix()` followed by `rlBegin(...)/rlEnd(...)`, which is exactly the kind of small-immediate-call pattern Dreamcast is least likely to tolerate well. 7

Another confirmed bottleneck is **raylib’s high-frequency 2D helper layer generating lots of textured quad work with helper-local state flips**. In `rshapes.c`, many helpers bind the default shapes texture, begin `RL_QUADS`, emit a few vertices, then restore texture state; the same file also falls back to plain triangles in some branches. This is convenient and portable, but on Dreamcast it means many tiny helper calls can generate repeated binds and small batches instead of a long run of homogeneous geometry. 8

At the PVR boundary, **submission order matters materially**. KOS explicitly states that if your data is already grouped by primitive/list type, direct submission is the fastest path; if it is not, the buffered path

copies data into main-RAM vertex buffers until the right list becomes active, which is slower. Since GLdc is accumulating list data and rebuilding headers as state changes, any renderer pattern that rapidly alternates opaque/translucent/state combinations is fighting both GLdc and the hardware. ⁹

A lower-confidence but plausible bottleneck is **texture-path conversion and copy overhead**, especially for dynamic textures, streaming uploads, or formats that do not line up well with the Dreamcast/PVR sweet spot. I would not rank this above draw submission for a typical 3D scene, but it is credible enough to keep on the list: a Dreamcast port report from the Xash3D work attributes meaningful frame-rate gains to GLdc-side `texture.c` and `fast_path` changes, specifically reducing copy overhead and fixing alignment issues. That is anecdotal rather than authoritative, but it is consistent with the rest of the code path. ¹⁰

What GLdc is best at and whether raylib is using it well

GLdc's best-exposed fast lane is the **array-draw submission path**. `glDrawArrays()` and `glDrawElements()` both go to `submitVertices()`, and from there `generate()` can use dedicated fast-path generators for quads and triangles, plus a dedicated indexed fast path. This is the path most worth feeding. If a renderer can hand GLdc large runs of array or indexed geometry in fast-path-compatible attribute layouts, it avoids the worst-case "tiny helper call" pattern. ¹¹

Raylib is only using that fast lane well in some classes of work. The helper-heavy 2D/procedural layer is visibly built around small `rlBegin / rlEnd` quad emission and texture toggling, which is a poor fit for Dreamcast. By contrast, mesh-like work can at least conceptually benefit from array-draw submission, because `rlgl` exposes array APIs and GLdc has explicit fast-array handling. So the Dreamcast-friendly direction is clear: push more of raylib's hot draw volume toward long-lived client-array submissions and fewer immediate-style helper emissions. ¹²

The subtle but important caveat is that **indexed geometry is not automatically better than de-indexed geometry in this stack**. On desktop GPUs, indexing often benefits from efficient post-transform reuse in hardware. In GLdc's fast indexed path, however, the current code still parses each index, fetches each attribute stream, transforms each referenced vertex, and copies the enabled attributes into submission memory. That means indexed geometry still saves source bandwidth and source-memory size, but it does not magically avoid CPU-side T&L or gathering costs. For hot sprite/quad workflows, de-indexed or pre-expanded streams that map cleanly onto array fast paths can be better. For larger meshes with good locality and reused source data, indexing can still be the right compromise. ⁶

Recommended optimization targets

Highest-priority target: route more raylib hot paths into GLdc array fast paths. The most valuable upstream-style change is a Dreamcast-specific or OpenGL-1.1-specific quad/sprite batch path for the helpers that dominate frame time: sprites, text, UI rectangles, shape fills, and billboards. The goal is not to special-case one game API, but to replace many tiny helper-local quad emissions with one interleaved client-array batch per texture/material run, flushed with `glDrawArrays / glDrawElements` in a format GLdc likes. This is **high win / medium effort / medium risk**. ¹³

Second target: reduce state/header churn. Because GLdc emits a fresh poly header whenever GPU state is dirty or a list starts empty, material and texture ordering matter more than they do on a less state-

sensitive backend. Sort or coalesce where the API semantics permit it, especially for opaque textured quads and shape/UI traffic. Even without full scene sorting, internal batching of consecutive same-texture/same-blend helper draws would pay off. This is **medium-high win / low-to-medium effort / low risk**. ⁴

Third target: broaden GLdc's fast-path eligibility and reduce per-vertex copying inside it. The code already has specialized fast generators, so the low-risk direction is to make raylib's common client-array layouts hit them more often. The next step after that is attribute-mask-specialized fast generators so that paths with no normals, no secondary UVs, or no colors do not keep paying generic copy logic. This is **high win / medium effort / medium risk**. ⁶

Fourth target: be selective about indexed vs de-indexed submission. For general mesh rendering, keep indexed geometry available because it still reduces source-memory footprint and is broadly compatible with client-array submission. But for the hottest quad/triangle-heavy paths, especially 2D and billboard-like work, pre-expanded de-indexed streams may be faster because they map directly onto the specialized array fast paths and avoid index parsing/gather. This is **medium win / low effort / low risk**, if done only for hot renderer-wide paths rather than for game-specific content. ⁶

Fifth target: investigate more direct PVR submission or tighter KOS integration for already-sorted opaque runs. KOS documents that direct submission is the fastest when data is already grouped correctly; buffered/out-of-order submission is slower. The architectural upside here could be large, but this is the first recommendation that becomes invasive and backend-specific. This is **potentially very high win / high effort / high risk**. ⁹

Sixth target: clean up texture-path conversions and copies, but only after submission work. This is unlikely to be the top frame-time issue for a normal static-asset scene, but it can matter for uploads, streaming, and mismatch-heavy formats. Community Dreamcast-port evidence suggests GLdc-side copy/alignment work does move the needle. This is **situational win / medium effort / low risk**.

Suggested minimal patches

A minimal patch with the best chance of broad payoff is to add a **Dreamcast-only helper batch inside raylib/rlgl for textured quads**. Conceptually, this is a tiny internal batcher for position/UV/color quads that appends helper draws into one interleaved client array until texture or blend state changes, then flushes with `glDrawArrays(GL_QUADS)` or an equivalent indexed variant. Guard it under a Dreamcast or OpenGL-1.1 backend macro so PC/web behavior stays unchanged. This directly attacks the repeated `rlSetTexture(...)`, `RL_QUADS`, small-helper pattern seen in raylib's helper layer while steering work into GLdc's better-handled array-draw path. ¹⁴

A second minimal patch is a **GLdc instrumentation pass** before any large refactor: count total `glDrawArrays` / `glDrawElements` calls, fast-path hits vs misses, headers emitted, state-dirty transitions, vertices transformed, and bytes copied in attribute gathering. The code structure already gives natural hook points: `submitVertices()`, `generate()`, `generateElementsFastPath()`, and `apply_poly_header()`. This is the cheapest way to turn the current source-based investigation into hard hardware numbers. ¹⁵

A third minimal patch is to **widen GLdc fast-path usage instead of inventing a new renderer**. Specifically: accept more common raylib client-array layouts as fast-path-eligible, add specialized fast generators for the common “position + UV + color” case, and skip copying normals/ST when those attributes are disabled. That keeps the API stable, keeps PC/web untouched, and improves the bottleneck that current source inspection points to most clearly. ⁶

A fourth minimal patch is a **state-coalescing pass in raylib helper code**. The low-risk version is simple: avoid helper-local texture disable/re-enable patterns when a sequence of same-texture draws can legally remain inside one internal batch; the more ambitious version is to let shapes/text share a common Dreamcast quad batch where ordering stays intact but flushes happen only when texture/blend/material actually changes. This is narrower than a full renderer rewrite and still attacks one of the most obvious raylib-side inefficiencies. ¹⁶

Open questions and limitations

This report is strongest on the array-draw and PVR-submission side because those paths were directly inspectable and well documented. The least complete area is the exact behavior of GLdc’s immediate-mode implementation and the precise conditions that set `ATTRIB_LIST.fast_path`; both matter, but the available public indexed views did not expose every relevant file cleanly.

I also would not present the ranking here as a substitute for hardware profiling. The source strongly suggests that submission overhead, header churn, and immediate-mode helper volume are the main renderer-wide problems, but the exact split between “raylib helper overhead” and “GLdc submission overhead” should still be confirmed on-device with counters and a few representative scenes.

¹ GitHub - psxdev/raylib4Dreamcast: Raylib port for Sega Dreamcast · GitHub

https://github.com/psxdev/raylib4Dreamcast?utm_source=chatgpt.com

² ⁷ raw.githubusercontent.com

https://github.com/raysan5/raylib/raw/refs/heads/master/src/rlgl.h?utm_source=chatgpt.com

³ raylib/src/rlgl.h at master · raysan5/raylib · GitHub

https://github.com/raysan5/raylib/blob/master/src/rlgl.h?utm_source=chatgpt.com

⁴ ⁵ ⁶ ¹⁰ ¹¹ ¹⁵ GLdc/GL/draw.c at master · Kazade/GLdc · GitHub

https://github.com/Kazade/GLdc/blob/master/GL/draw.c?utm_source=chatgpt.com

⁸ ¹² ¹³ ¹⁴ ¹⁶ raylib/src/rshapes.c at master · raysan5/raylib · GitHub

https://github.com/raysan5/raylib/blob/master/src/rshapes.c?utm_source=chatgpt.com

⁹ KallistiOS: Scene Submission

https://kos-docs.dreamcast.wiki/group__pvr__scene__mgmt.html?utm_source=chatgpt.com